

Promise (Basics)

Analogy

Ek singer (producing code) apna gaana baad me release karega. Fans (consuming code) ne subscription list me naam likha hai. Gaana aate hi sabko mil jaata hai. Bura ho (fire lag gayi) to bhi sabko khabar mil jaati hai.

Promise = wo subscription list, jo producing code aur consuming code ko jodti hai.

Promise banana

javascript

```
let promise = new Promise(function (resolve, reject) {  
  // executor: kaam yahan hota hai  
});
```

- Executor **apne aap turant** chalta hai.
- `resolve(value)`: kaam successfully ho gaya.
- `reject(error)`: error aa gaya. (`Error`) object dena behtar hai.)

Promise ke internal state:

- `pending` (shuru me) -> `fulfilled` (resolve hone par) ya `rejected` (reject hone par).
- `result`: `undefined` se badal kar value ya error ban jaata hai.
- `state` aur `result` seedhe access nahi kar sakte, `.then/.catch/.finally` se hi milte hain.

javascript

```
new Promise(resolve => setTimeout(() => resolve("done"), 1000));  
new Promise( (_, reject) => setTimeout(() => reject(new Error("Whoops!")), 1000)
```

Sirf ek result ya error hota hai. Baad ki `resolve/reject` calls **ignore** ho jaati hain.

Pehle se ready value ho to turant `resolve(123)` bhi kar sakte ho.

Consumers: `then` aur `catch`

javascript

```
promise.then(  
  result => { /* success */ },  
  error => { /* failure */ }  
);
```

- Sirf success chahiye: `.then(f)`.
- Sirf error chahiye: `.catch(f)` (yaani `.then(null, f)`).

`finally`

Cleanup ke liye (loading indicator band karna). Hamesha chalta hai.

- Ise **koi argument nahi** milta.
- Result ya error ko **aage pass** kar deta hai.
- Iska return value ignore hota hai (par `throw` kare to error aage jaata hai).

javascript

```
promise.finally(() => stopLoading()).then(show, showError);
```

Handler baad me bhi jod sakte ho

Agar promise pehle hi settle ho chuka hai, aur tum baad me `.then` lagao, to handler **turant** chalta hai. Ye real-life subscription list se zyada flexible hai.

Example: `loadScript` promise ke saath

javascript

```
function loadScript(src) {  
  return new Promise((resolve, reject) => {  
    let script = document.createElement("script");  
    script.src = src;  
    script.onload = () => resolve(script);  
    script.onerror = () => reject(new Error(`Load error: ${src}`));  
    document.head.append(script);  
  });  
}  
  
loadScript("a.js").then(script => console.log("loaded"), err => console.log(err
```

Promises vs Callbacks

Promises

Natural order: pehle call, phir `.then` me batao kya karna hai

Ek promise par kai `.then` laga sakte ho

Callbacks

Call karne se pehle hi callback dena padta hai

Sirf ek callback

Tasks

- `resolve(1)` ke baad `resolve(2)` karo to output `1` hi aata hai (dobara resolve ignore).
- `delay(ms)` (promise wala `setTimeout`):

javascript

```
function delay(ms) {  
  return new Promise(resolve => setTimeout(resolve, ms));  
}  
delay(3000).then(() => console.log("3 sec baad"));
```

Yaad rakho

- `new Promise((resolve, reject) => ...)`
- Ek hi outcome, baaki ignore.
- `.then`, `.catch`, `.finally` se result lo.